37 SQL CHEATSHEET — COMPLETE REFERENCE WITH EXAMPLES

All examples below assume the `students` table from theory/36-postgres-setup-and-crud.txt:

```
students
--------
id          SERIAL PRIMARY KEY
name        VARCHAR
age         INT
email       VARCHAR UNIQUE
course      VARCHAR
created_at  TIMESTAMP
```

...plus a second table `courses` (id, title, credits) used further down for JOIN examples, since JOINs need at least two related tables to make sense.

1. CREATING / CHANGING TABLES

CREATE TABLE:

```sql
CREATE TABLE students (
  id          SERIAL PRIMARY KEY,          -- auto-incrementing integer, like Mongo's _id but sequential
  name        VARCHAR(100) NOT NULL,
  age         INT NOT NULL,
  email       VARCHAR(255) UNIQUE NOT NULL,
  course      VARCHAR(100),                -- no NOT NULL -> optional, like course?: string in the DTO
  created_at  TIMESTAMP DEFAULT NOW()
);
```

ALTER TABLE (change structure after creation):

```sql
ALTER TABLE students ADD COLUMN gpa NUMERIC(3,2);      -- add a column
ALTER TABLE students DROP COLUMN gpa;                  -- remove a column
ALTER TABLE students RENAME COLUMN course TO major;    -- rename a column
ALTER TABLE students ALTER COLUMN age SET NOT NULL;    -- change a constraint
```

DROP TABLE (deletes the table and all its data — irreversible):

```sql
DROP TABLE students;
```

2. INSERT — CREATE ROWS

```sql
INSERT INTO students (name, age, email, course)
VALUES ('Rahul', 20, 'rahul@example.com', 'NID');
```

Insert multiple rows in one statement:

```
INSERT INTO students (name, age, email, course) VALUES
  ('Priya', 21, 'priya@example.com', 'CS'),
  ('Arjun', 22, 'arjun@example.com', 'ECE');
```

Get the inserted row back (Postgres-specific, very useful — this is what TypeORM's .save() relies on under the hood):

```
INSERT INTO students (name, age, email)
VALUES ('Rahul', 20, 'rahul@example.com')
RETURNING *;
```

3. SELECT — READ ROWS

All columns, all rows:

```
SELECT * FROM students;
```

Specific columns only:

```
SELECT name, email FROM students;
```

WHERE — filter rows:

```
SELECT * FROM students WHERE age > 20;
SELECT * FROM students WHERE course = 'CS';
SELECT * FROM students WHERE age BETWEEN 20 AND 25;
SELECT * FROM students WHERE name LIKE 'R%';        -- starts with R
SELECT * FROM students WHERE name ILIKE 'r%';       -- case-insensitive version
SELECT * FROM students WHERE course IN ('CS', 'ECE'); -- matches any of these
SELECT * FROM students WHERE course IS NULL;        -- no direct = NULL — must use IS NULL
SELECT * FROM students WHERE age > 20 AND course = 'CS';
SELECT * FROM students WHERE age < 18 OR age > 60;
SELECT * FROM students WHERE NOT course = 'CS';
```

ORDER BY — sort:

```
SELECT * FROM students ORDER BY age ASC;    -- ascending (default)
SELECT * FROM students ORDER BY age DESC;   -- descending
SELECT * FROM students ORDER BY course, age DESC;  -- multiple keys
```

LIMIT / OFFSET — pagination (this is your "page 2 of results" tool):

```
SELECT * FROM students ORDER BY id LIMIT 10;           -- first 10
SELECT * FROM students ORDER BY id LIMIT 10 OFFSET 20;  -- rows 21-30 ("page 3" at page size 10)
```

DISTINCT — unique values only:

```
SELECT DISTINCT course FROM students;
```

AS — rename a column in the result (alias):

```sql
SELECT name AS student_name, age AS student_age FROM students;
```

4. UPDATE — MODIFY ROWS

```sql
UPDATE students SET age = 22 WHERE id = 1;
UPDATE students SET age = 22, course = 'CS' WHERE id = 1;  -- multiple columns at once
```

DANGER: forgetting WHERE updates EVERY row in the table:

```sql
UPDATE students SET course = 'CS';   -- <- sets EVERY student's course to CS. Always double check WHERE before running an UPDATE.
```

Get the updated row back (mirrors Mongoose's { new: true }):

```sql
UPDATE students SET age = 22 WHERE id = 1 RETURNING *;
```

5. DELETE — REMOVE ROWS

```sql
DELETE FROM students WHERE id = 1;
```

Same danger as UPDATE — no WHERE deletes everything:

```sql
DELETE FROM students;   -- <- empties the whole table, rows are gone (not the table structure — for that use DROP TABLE or TRUNCATE)
```

TRUNCATE — faster way to empty a table completely (skips row-by-row logging):

```sql
TRUNCATE TABLE students;
```

6. AGGREGATE FUNCTIONS — SUMMARIZING DATA

```sql
SELECT COUNT(*) FROM students;                  -- how many rows
SELECT COUNT(*) FROM students WHERE course = 'CS'; -- how many match a filter
SELECT AVG(age) FROM students;                  -- average
SELECT MIN(age), MAX(age) FROM students;         -- smallest / largest
SELECT SUM(age) FROM students;                   -- total (less useful for age, but same idea as summing a price column)
```

GROUP BY — aggregate PER group instead of over the whole table:

```sql
SELECT course, COUNT(*) AS student_count
FROM students
GROUP BY course;
-- "how many students are in each course" — one row per distinct course

SELECT course, AVG(age) AS avg_age
FROM students
GROUP BY course;
```

HAVING — filter groups AFTER aggregating (WHERE can't do this — WHERE filters rows BEFORE grouping, HAVING filters the grouped results):

```
SELECT course, COUNT(*) AS student_count
FROM students
GROUP BY course
HAVING COUNT(*) > 5;   -- only courses with more than 5 students
```

7. JOINS — COMBINING RELATED TABLES

Setup for these examples — a second table, and a foreign key linking students to it:

```
CREATE TABLE courses (
  id      SERIAL PRIMARY KEY,
  title   VARCHAR(100),
  credits INT
);

ALTER TABLE students ADD COLUMN course_id INT REFERENCES courses(id);
-- course_id is a FOREIGN KEY: Postgres now REJECTS any student row
-- whose course_id doesn't match a real courses.id. This is the
-- referential-integrity guarantee Mongo doesn't give you for free.
```

INNER JOIN — only rows that match in BOTH tables:

```
SELECT students.name, courses.title
FROM students
INNER JOIN courses ON students.course_id = courses.id;
-- a student with no course_id (NULL), or pointing at a non-existent
-- course, would NOT appear in these results
```

LEFT JOIN — all rows from the left table, matched data from the right table where it exists, NULL where it doesn't:

```
SELECT students.name, courses.title
FROM students
LEFT JOIN courses ON students.course_id = courses.id;
-- every student appears, even ones with no course_id — courses.title
-- is just NULL for them
```

RIGHT JOIN — mirror of LEFT JOIN (all rows from the right table):

```
SELECT students.name, courses.title
FROM students
RIGHT JOIN courses ON students.course_id = courses.id;
-- every course appears, even ones with zero enrolled students
```

FULL OUTER JOIN — everything from both sides, matched where possible:

```
SELECT students.name, courses.title
FROM students
FULL OUTER JOIN courses ON students.course_id = courses.id;
```

Table aliases (shorthand, very common in real queries):

```sql
SELECT s.name, c.title
FROM students s
JOIN courses c ON s.course_id = c.id;
```

8. SUBQUERIES — A QUERY INSIDE A QUERY

```sql
-- students in courses worth more than 3 credits
SELECT * FROM students
WHERE course_id IN (
  SELECT id FROM courses WHERE credits > 3
);

-- students older than the average age
SELECT * FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

9. CTEs (WITH clause) — NAME A SUBQUERY FOR READABILITY

```sql
WITH high_credit_courses AS (
  SELECT id FROM courses WHERE credits > 3
)
SELECT * FROM students
WHERE course_id IN (SELECT id FROM high_credit_courses);
-- same result as the subquery version above, but readable in steps,
-- and reusable multiple times in one query
```

10. CONSTRAINTS — RULES POSTGRES ENFORCES FOR YOU

```sql
NOT NULL     -- column can't be left empty
UNIQUE       -- no two rows can have the same value (email UNIQUE)
PRIMARY KEY  -- unique + not null + identifies the row (usually id)
FOREIGN KEY  -- value must exist in another table's column (course_id REFERENCES courses(id))
CHECK        -- custom rule, e.g.:
               ALTER TABLE students ADD CONSTRAINT age_check CHECK (age > 0);
DEFAULT      -- value used if none provided, e.g. created_at TIMESTAMP DEFAULT NOW()
```

This is the "hard schema" difference from theory/36 — these rules are enforced by the DATABASE ITSELF, not just by your app's validation pipe. Even a raw SQL client bypassing your NestJS app entirely can't violate them.

11. INDEXES — SPEEDING UP LOOKUPS

```sql
CREATE INDEX idx_students_email ON students(email);
-- speeds up WHERE email = '...' / JOINs on email, at the cost of
-- slightly slower INSERTs (the index has to be updated too)

-- UNIQUE already auto-creates an index, so email here already has
-- one from the UNIQUE constraint above — this is just showing the
-- explicit syntax for a column that ISN'T already unique, e.g.:
CREATE INDEX idx_students_course ON students(course);
```

12. TRANSACTIONS — GROUP MULTIPLE STATEMENTS, ALL-OR-NOTHING

```
BEGIN;
  UPDATE students SET course = 'CS' WHERE id = 1;
  UPDATE courses SET credits = credits - 1 WHERE id = 5;
COMMIT;       -- both changes are saved together
```

-- or, if something goes wrong partway through:

```
BEGIN;
  UPDATE students SET course = 'CS' WHERE id = 1;
  -- something fails here
ROLLBACK;     -- undoes everything since BEGIN, as if neither ran
```

This atomicity guarantee (all statements succeed together, or none do) is a core reason relational DBs get chosen for financial/inventory systems — Mongo has multi-document transactions too, but they're opt-in and less central to how the DB is normally used.

13. WINDOW FUNCTIONS — AGGREGATE WITHOUT COLLAPSING ROWS

GROUP BY collapses many rows into one per group. A window function computes an aggregate but still shows every individual row:

```
SELECT name, age, course,
       AVG(age) OVER (PARTITION BY course) AS avg_age_in_course
FROM students;
-- every student row stays, but each one also shows their course's
-- average age next to it

SELECT name, age,
       RANK() OVER (ORDER BY age DESC) AS age_rank
FROM students;
-- ranks every student by age, oldest = rank 1, without grouping anything away
```

14. QUICK MONGOOSE → SQL PHRASE TRANSLATION

.find()	→ SELECT *
.find({ course: 'CS' })	→ SELECT * WHERE course = 'CS'
.find().sort({ age: -1 })	→ SELECT * ORDER BY age DESC
.find().limit(10).skip(20)	→ SELECT * LIMIT 10 OFFSET 20
.findById(id)	→ SELECT * WHERE id = $1
.findByIdAndUpdate(id, dto)	→ UPDATE ... WHERE id = $1
.findByIdAndDelete(id)	→ DELETE FROM ... WHERE id = $1
.countDocuments()	→ SELECT COUNT(*)
.find({ age: { $gt: 20 } })	→ WHERE age > 20

.find({ age: { $in: [20,21] }})	→ WHERE age IN (20, 21)
populate() (join a referenced doc)	→ JOIN

See also: **theory/36-postgres-setup-and-crud.txt** (TypeORM/NestJS side of this), **theory/30-mongodb-connection.txt**, student.schema.ts